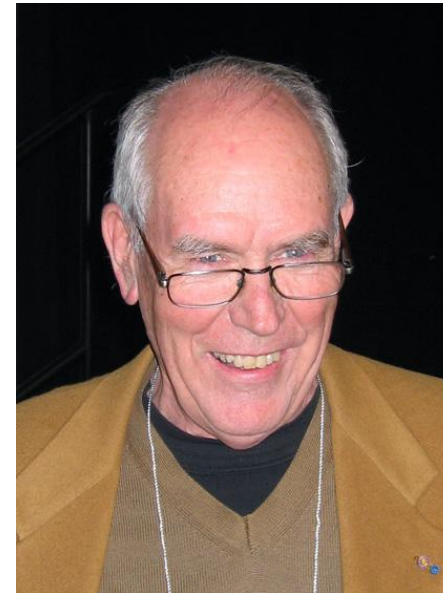


Objectoriented Programming

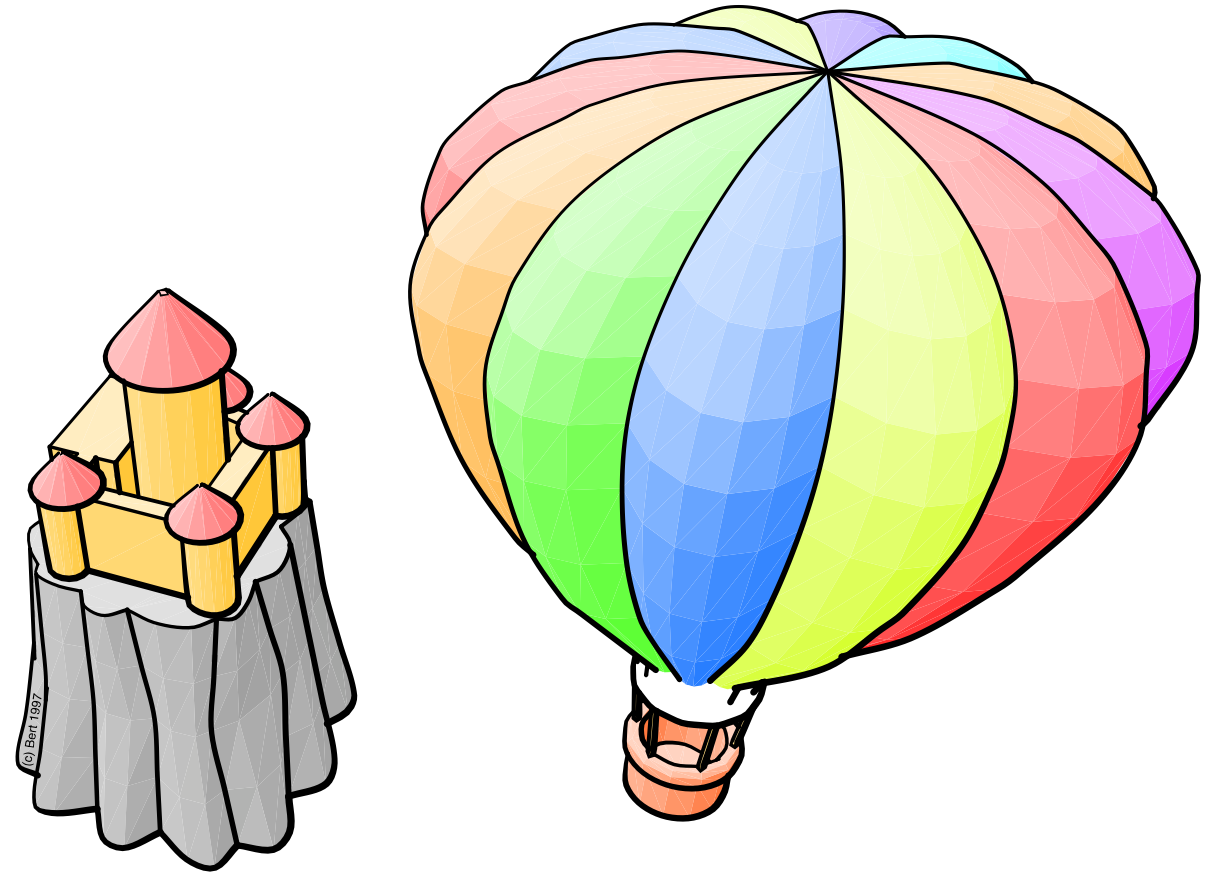
1. History
2. What is a class?
3. Motivation
4. Objects and attributes
5. Classes in Python
6. Creating your own class
 1. Defining a Class
 2. Member Methods
 3. magic functions
7. Definitions

History

- The terminology of objects and oriented in the modern sense stems from the late 1950s to early 1960s at the MIT
- Pioneers in this field were Alan Kay and Ivan Sutherland, who in 1966 and 1963 coined the terms objects and instances

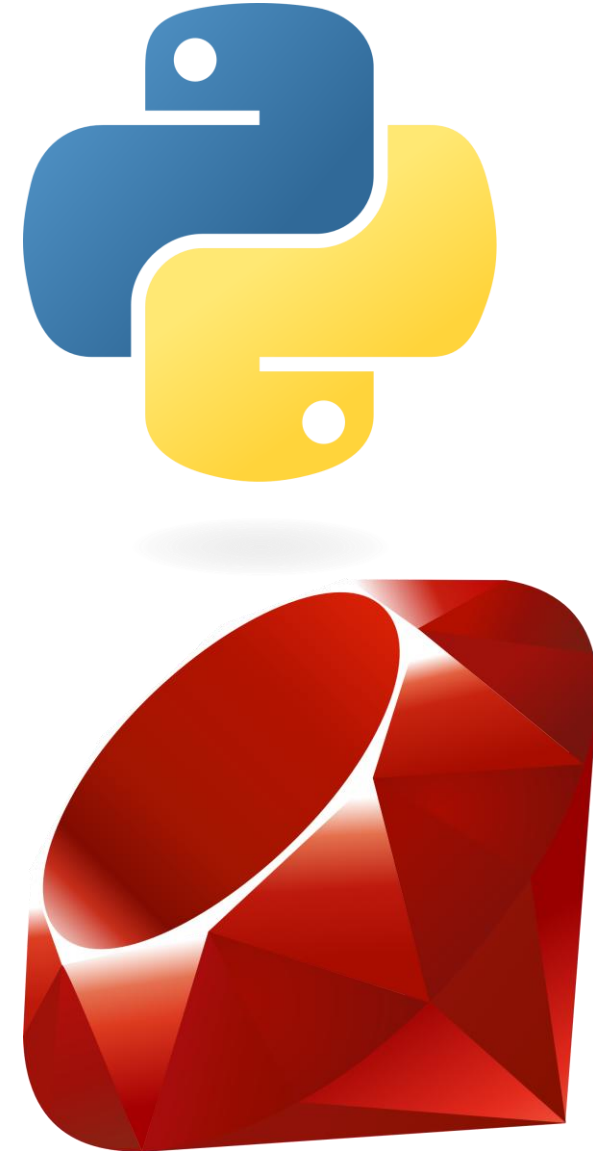


- One of the first object-oriented programming languages was Smalltalk, which was developed at the MIT mainly for educational purposes
- It was dynamically typed and an Interpreter language similar to Python nowadays



- In the early and mid 1990s object-oriented programming became the dominant paradigm
- This stemmed from programming languages like C++ or Delphi becoming widely available
- Furthermore the rising popularity of graphical user interfaces enhanced this dominance, since these GUIs heavily rely on object oriented programming techniques

- Nowadays programming languages like Python and Ruby emerged, which are mainly object-oriented but also support other programming paradigm
- The probably most commercially important programming language in this field is Java



What is a class?

- Integer is a data type that contains a number with which you can do several different things
- A list is a datastructure that is used to collect multiple elements and we have several different methods we can use on them, for example the methods `append()` and `extend()`
- A class is a user-defined data type or datastructure which contains information und defines the operations, that means the methods, which can be done with him
- This is used to combine all code of one object in single definition / block of code

- A class is the blue-print for creating objects
- It combines the following:
 - ▶ all values saved in an object, the so called attributes
 - ▶ all functions useable on an object, the so called member methods
 - ▶ all functions that tell how an object interacts with built-in functions and operators, the so called magic functions

Motivation

- They allow for easy maintenance of source code
 - ▶ All functions are in one place
 - ▶ You can make changes to the behavior of your objects in a simple manner
 - ▶ You can avoid most unwanted side results of maintaining your code
- They allow you to reuse your code in an easy manner
 - ▶ You have one script in which you define your class
 - ▶ Later on you import the class to other scripts and use the methods in there

- First you create a script that contains your class (left side)
- This file should have the same name as your class but in all lowercase letters (plants.py)
- Than you create a second file (garden.py for example) and in there you import your class (right side)
- Now you can create objects of your class using the name of your class

```
class Plants:
    def __init__(self):
        ...
```

```
from plants import Plants

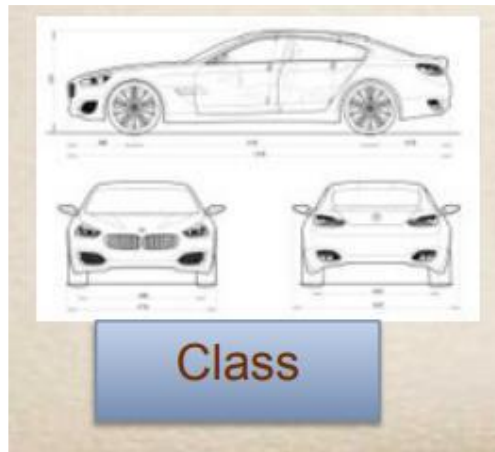
pear = Plants()
```

- Once you created your class you need to save it somewhere
- You can save it in the same directory as your other Scripts (garden.py) that make use of the class
- Alternatively you save it in the directory where all other Python modules are placed
 - ▶ Under windows (using Thonny) you can find all modules in the following directory:
C:\Users\User\AppData\Roaming\Python\Python310\site-packages
 - ▶ Another potential folder under windows is the following: C:\python34\Lib\site-packages\

- A third method is to save your classes in a special directory and use the module sys to help with the import
- See the following link for a detailed description:
 - ▶ <https://www.geeksforgeeks.org/python-import-module-outside-directory/>

Objects and Attributes

- A class is the definition of something you could call it a Blue-Print
- An object is one instance of this class, that means one variable we define to be of the type of our class



- An object can be a variable, a data structure or a function
- A class combines variables and functions in a comprehensive definition and as soon as we create a variable of the type of our class it has access to all these functions/methods as well
- A simple example are lists where list itself is the class and each list we define in our script is then an object of the class/type list

- Variables we define for our class are called attributes and can be used to store data in our object
- In python they are called via `object.attribute`
- For example our class BMW CS could contain attributes called color or kilometers and than we could assign values ot these attributes with the following lines of code:

```
BMW1.color = "red"
```

```
BMW2.kilometers += 1
```

Member Methods

- We can define normal functions within a class and these functions are then called member methods of our class
- We already saw such methods in action when we were talking about data structures, where we used methods like `append()` or `extend()` on our lists
- The general usage is as follows: `object.method()`
- A Member-Method works on the object it is called on

- Special Member-Method that is called when a new object of the class is generated, that means when we define a new variable to be of the type of our class
- Has a very specific name, depending on the programming language and is the first thing we should define in our class
- We can define parameters for our constructor to use them on the creation of a new object, for example to assign values to our attributes

- Can tell how the Object should behave in conjunction with
 - Operators, print, casting functions, len, ...
- A set of predefined functions with the general structure `__name__(self, otherparas)`:
- Each class is assigned a standard version of these magic functions as soon as we define it and later on we can overwrite the standard version by defining them inside our own class
- <https://rszalski.github.io/magicmethods/>

Classes in Python

- In python everything is a class, even a simple integer variable, as we saw when we were using the built-in function `type`, which always returned statements like `<class 'int'>`
- With the help of `dir(object)` all available Member-Methods as well as attributes and magic functions of an object can be displayed

- Python contains a built-in function called `dir()`
- This function allows you to look up all methods/attributes your object has access to
- It returns a list of strings and each element in this list is the name of one method/attribute of your object
- The main usage is in combination with `print`:

```
list1 = [5, 6, 7]  
  
print(dir(list1))
```

- For the object list1 we saw on the previous slide the print out in the terminal would look as follows:
 - ▶ ['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
- At the end of the list you see methods like append(), extend() or sort()

- At the beginning you see elements like ‘__add__’, ‘__eq__’ or ‘__str__’ that start and end with two underscores “_”
- These are the so-called magic functions and we will discuss them in detail later on
- However if you look at object of other classes, like dictionaries or tuples you will see many magic functions with the same names there as well

Creating your own class

- When defining our own class we need to think about different things:
 - Which attributes do we need?
 - What member methods do we want to define?
 - Do we need to define magic functions?
- Of course we can include these things step by step after we defined our class in general, nevertheless it's a good idea to create an outline of your class before you type
- This can help you to determine how you need to define your attributes, which kind of checks you need to implement and so on

- Looks similar to a function definition but we use the keyword `class` instead of *def*
- Afterwards you just write down the name of your class and end the line with a colon „:“
- You don't need any brackets in this line
- class names should start with an upper case letter and if you use multiple words, simply write them after each other but each word should start again with an upper case letter:

```
class Date:
```

- As we saw the line of our class definition ended with a :, that means we need at least one additional line of code afterwards
- Here we want to define a class called Date that can save the day, month and year
- We could do this by simply defining the attributes day, month and year after the class definition:

```
class Date:  
    day = 0  
    year = 0  
    month = 0
```

- What we did on the previous slide is called defining class attributes
- These attributes will contain the same value for all objects of our class and thus are defined directly after the class definition
- In most cases we are more interested in so called instance attributes, that means attributes which differ from object to object
- To achieve such a thing we need to create a constructor for our class with the help of the magic function `__init__`

- Now we can define the constructor for our class
- This is done with the help of a simple function definition but the name of the function ALWAYS has to be `__init__` and it ALWAYS needs the parameter `self`
- The attributes are then defined inside the function with the point notation `self.attributename:`

```
class Date:  
    def __init__(self):  
        self.day = 0  
        self.year = 0  
        self.month = 0
```

- In most cases it doesn't make sense to define our attributes with pre-defined values (even though we can do so)
- Instead we want to be able to hand over values to our object on construction, that means on definition, of it
- We can do this by adding additional parameters to our function `__init__`:

```
class Date:  
  
    def __init__(self, arg_day, arg_month, arg_year):  
        self.day = arg_day  
        self.year = arg_year  
        self.month = arg_month
```

- When creating a class you define attributes only in the constructor (or after the class definition) and nowhere else
- If attributes should be assigned values on construction of an object you always need the parameters as a form of helper variables to assign values to your attributes

- We can use the keyword self to represent one instance / object of our class
- It is used in the definition of methods for our class to tell python that this method works on the corresponding object
- Therefore we use it as the first parameter in the signature of our function inside of our class:

```
def __init__(self,parameters):
```

- Furthermore we use it to get access to the values of the attributes of our class inside of our class:

```
print(self.attribute)
```

- We could use other terms than self, for example in_place_of_self, and our methods would still work:

```
def __init__(in_place_of_self):
```

- BUT you should stick to self to improve readability and it is also good programming practice
- In conclusion, self is a placeholder for the name of the object we defined in our script

Class attributes

- Python differentiates between public, protected and private attributes of classes
- We can define these different types of attributes with the help of underscores, when naming our attributes
- If we use no underscores before the name of an attribute, we create a public attribute that we can change anywhere in our script:
 - ▶ `self.day = arg_day` ➔ public can be changed

- If we use one underscore before the name of an attribute and after the point, we create a protected attribute:
 - `self._day = arg_day` → protected, changings aren't desireable
- These attributes work just like a public attributes, but you should try to avoid changing their values outside of your class definition
- They mostly used to tell other programmers how you intend them to work with these attributes
- I would refrain from using protected attributes, since they only confuse you

- The last type of attributes are the private attributes
- We define them by using two underscores between the point and the name of our attribute
 - `self.__day = arg_day` ➔ private can't be changed
- These attributes can only be accessed inside our class definition and therefore are secured against any random changes in our script
- Please keep in mind that the underscores are part of the name afterwards

- Generate your own class called “Gene” that has the following five attributes:
 - ▶ `__Name`: a string (private)
 - ▶ `__Nr_Nucleotide`: a positive whole number (private)
 - ▶ `__Nr_ReadingFrame`: a positive whole number (private)
 - ▶ `Nucleotide`: a positive whole number (public)
 - ▶ `ReadingFrame`: a positive whole number (public)
- All attributes should be given as arguments/parameters to the constructor of the class if a new object is generated

Member methods in Python

- Member methods are functions we define inside our class definition
- They allow us to work with the attributes of our object or our object itself to run specific calculations
- We can name these methods as we want, but for some tasks there are member methods which have specific names
- Especially when working with private attributes
- Here we need methods to access the value of a private attribute (getter) or to change the value of a private attribute (setter)

- To read out the value of private attributes you always need to define a so called getter method
- Of course these methods need the parameter `self`, to be able to access the attribute of the current object
- Inside the method we only have a simple return statement, that hands back the corresponding private attribute
- We can name these getter methods as we want but it is good programming practice to name them `get_attributename`:

```
def get_displacement(self):  
    return self.__displacement
```

- To change the value of a private attribute we need to define a specific member method, that allows us to do so
- These methods are called Setter and while we can name them again as we want it is good programming practice to name them `set_attributename`
- These methods of course need the parameter `self` as well as at least one additional parameter that contains the new value for our private attribute:

```
def set_displacement(self, cubic):
```

- The main usage for these Setter methods is to make sure that the new value of the corresponding attribute fulfills certain criteria
- For example we want to accept only positive numbers for an attribute called `__displacement`:

```
def set_displacement(self, cubic):  
    if (cubic <= 0):  
        print("Error: Negative value for displacement!")  
    else:  
        self.__displacement = cubic  
        print("displacement was changed")
```

- Expand the class from the first exercise by adding several methods to it.
 - ▶ One `getter()` for each private attribute defined in the class
 - ▶ One `setter()` method for each private attribute in the class
 - ▶ A method `print_state()` that prints out the name of the Gene the used Nucleotide and the used ReadingFrame

Magic functions

- Earlier you saw that many classes have methods with the same name, that start and end with an underscore “_”
- These methods are called magic functions
- They tell Python how an object of the class:
 - ▶ interacts with functions like `print()` or `len()`
 - ▶ works in combination with comparison operators like “==” or “>”
 - ▶ deals with mathematical operations like “+”
 - ▶ works with keywords like “in”
 - ▶ can be used by loops
 - ▶ ...

- Each class is assigned these magic function automatically when it's defined
- You don't need to define these methods on your own
- However you can define them on your own to overwrite the default definitions that were assigned to your class

- There is a built-in method for printing out the contents of an object of your own class
- This method is called the string-method and it is one of the so called magic functions that are defined for each class we create
- We can overwrite the general behavior of this method by defining it inside our class definition:

```
def __str__(self):  
    return ...
```

- The Variable you return with this method ALWAYS has to be a String

- A second magic function that can be useful is the function `__len__`
- This magic function tells Python how your object interacts with the built-in function `len()`
- The general definition of this magic function is similar to the one for `__str__`

```
def __len__(self):  
    return ...
```
- The main difference between `__str__` and `__len__` is that the latter always needs to return an integer number

- Add a String-method to your class, so that the contents of your objects can be printed out in a good manner
- Add the function `__len__` to your class so that the length of your gene can be determined

Exercise Plants

Definitions

- A variable we declare with the help of our class becomes an **Object** of the class
- Variables defined in classes are called **Attributes**
- **Member Methods** are functions we define in our class definition
- We can specify how an object of our class interacts with built-in functions like print with the help of **Magic Functions**
- Attributes that can be changed anywhere in the script are **Public Attributes**
- If an attribute can be accessed only inside the class definition it's called **Private Attribute**
- Member methods that give us access to the value of a private attribute are called **Getter**

- We can change the value of private attributes with the help of **Setter** methods

1. https://en.wikipedia.org/wiki/Object-oriented_programming (Last edited: 02.04.2023. Last visited: 11.04.2023)
2. [https://en.wikipedia.org/wiki/Object_\(computer_science\)](https://en.wikipedia.org/wiki/Object_(computer_science)) (Last edited: 28.01.2023, Last visited: 11.04.2023)
3. [https://en.wikipedia.org/wiki/Attribute_\(computing\)](https://en.wikipedia.org/wiki/Attribute_(computing)) (Last edited: 21.02.2023, Last visited: 11.04.2023)
4. [https://en.wikipedia.org/wiki/Method_\(computer_programming\)](https://en.wikipedia.org/wiki/Method_(computer_programming)) (Last edited: 09.02.2023, Last visited: 11.04.2023)
5. <https://pynative.com/python-class-variables/> (Last edited: 07.02.2022, Last visited: 05.01.2023)
6. <https://www.geeksforgeeks.org/self-in-python-class/> (Last edited: 01.11.2022, Last visited: 11.04.2023)